GUIDE 38 · OPERATE

Allow a browser-based frontend to call the API directly

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=38-allow-a-browser-frontend-cors. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a web page on another origin - `https://app.example.com` calling `https://api.example.com` - that can call QueryAPIGate from the browser and read the responses, and an understanding of what that does and doesn't protect.

The problem

Browsers block a page's JavaScript from reading responses from a different origin unless the server allows that origin with CORS headers. Calls that send `X-API-Key` also get a "preflight" `OPTIONS` request first, which the server must answer. QueryAPIGate does neither until you turn it on - verified in Chromium: a page on `http://localhost:8099` calling the API on `http://127.0.0.1:5055` failed with `TypeError: Failed to fetch`.

Step 1: List your frontend's origin

```shell
QUERYAPIGATE_CORS_ORIGINS=https://app.example.com queryapigate serve
```

Several origins are comma-separated: `https://app.example.com, https://admin.example.com`. An origin is scheme, host and port - `https://app.example.com` doesn't cover `http://app.example.com` or `https://app.example.com:8443`.

Step 2: Call it from the page

```javascript
const res = await fetch('https://api.example.com/q/orders_by_status?status=paid&page_size=2', {
  headers: { 'X-API-Key': key },
});
const rows = await res.json();
res.headers.get('X-Has-More');                 // pagination
res.headers.get('X-RateLimit-Key-Remaining');  // this key's own rate limit
res.headers.get('X-Request-Id');               // for support requests
```

Verified in Chromium, with the page's origin listed:

```json
{"status": 200, "rows": 2, "hasMore": "true", "keyRemaining": "99", "requestId": "b77f52110199"}
```

and with a *different* origin listed instead (`https://app.example.com`), the same page got `Failed to fetch` again.

What CORS lets through:

- **Methods:** `GET`, `POST`, `PATCH`, `DELETE`. **Request headers:** `Content-Type`, `X-API-Key`, `Authorization`, `X-Request-Id`.
- **Response headers a page may read:** `X-Page`, `X-Page-Size`, `X-Has-More`, `X-RateLimit-*` (server-wide and per-key), `Retry-After`, `X-Request-Id`, `X-Cache`, `ETag`, `Deprecation`, `Link`.
- **No cookies** - credentials are the header you send.
- The preflight answer is cached by the browser for 10 minutes.

CORS isn't authentication

CORS tells *browsers* which pages may read responses. It doesn't stop anything else - `curl`, a server, a script - from calling the API. The API key is what protects it. Two consequences:

Any key in a page's JavaScript is public. Anyone who opens the page can read it in the developer tools. So the key a browser app uses must be one you'd accept anyone using:

- scoped to exactly the saved queries the page needs (guide 14) - `connections: []`, so no ad-hoc SQL;
- read-only, with a rate limit;
- or, better for an app with logins: no shared key at all - each user sends their own token as `Authorization: Bearer ...`, and a `from_claim` parameter limits each user to their own rows. See Signed-in users (JWT).

`*` is for public data only. `QUERYAPIGATE_CORS_ORIGINS=*` allows every website. It worked in the test above, as expected - and it means any page anyone visits can call your API through their browser. The real trap is `*` on a server **with no API key configured**: then any website a person visits can query your databases through *their* browser - including a QueryAPIGate on their own laptop or office network, which the website itself could never reach. The Console's **Alerts** flag a server with no key (`open_server`) as critical. Don't combine the two.

Behind a proxy

If your reverse proxy also adds CORS headers, let only one of them do it - browsers reject a response with two `Access-Control-Allow-Origin` headers. Usually that means leaving it to QueryAPIGate and keeping the proxy out of it.

The `queryapigate events` server reads the same `QUERYAPIGATE_CORS_ORIGINS`, so a page can open a live-events stream with `fetch()` too (guide 28).

Next steps

- Give an external partner access to exactly one query - the right shape for a browser key.
- Give each user of your app their own private activity feed - JWT for per-user access.
- Rate-limit a key.